

CESIZen

Choix techniques & comparatif d'architectures

Projet	CESIZen — Application de santé mentale
Référentiel	Concepteur Développeur d'Applications (CDA)
Bloc	Bloc 2 — Développer et tester les applications
Auteur	Tim LOPES
Version du document	1.0

1. Contexte et objectif du document

Ce document a pour but de présenter et de justifier les choix techniques retenus pour la réalisation du projet **CESIZen**, application grand public de santé mentale spécifiée par le CESI dans le cadre du titre Concepteur Développeur d'Applications (CDA). Il s'inscrit dans le Bloc 2 « Développer et tester les applications informatiques ».

Le projet comporte trois sous-applications interdépendantes : une **API REST** exposant les données métier, une **application mobile** destinée aux utilisateurs finaux et un **back-office Web** permettant aux administrateurs de gérer la plateforme. Le cahier des charges impose la réalisation de modules suivants :

- Gestion des comptes utilisateurs (création, modification, suppression, RGPD)
- Gestion d'*Informations* sur la santé mentale (contenus éditoriaux côté admin, consultation côté utilisateur)
- Tracker d'émotions (journal personnel + rapport agrégé) — module au choix
- Bonus : exercices de cohérence cardiaque (rythmes 7-4-8, 5-5, 4-6)

Le présent document compare trois architectures techniques candidates et explicite la solution finalement retenue, en s'appuyant sur des critères d'efficacité, de coût, de courbe d'apprentissage et de pérennité.

2. Critères d'évaluation

Pour comparer objectivement les architectures, six critères principaux ont été définis. Chaque critère est noté de **1 (faible)** à **5 (excellent)** dans le tableau de synthèse plus loin.

Critère	Description
Productivité	Vitesse de mise en œuvre et maturité de l'écosystème (CLI, scaffolding, ORM...).
Mobile cross-platform	Capacité à livrer une expérience native iOS et Android avec un seul codebase.
Maintenabilité	Lisibilité, architecture, séparation des couches, facilité d'évolution.
Performance	Rapidité d'exécution côté serveur ET côté client. Réactivité de l'UI mobile.
Coût	Coût total (licences, hébergement, formation des développeurs).
Documentation & communauté	Qualité de la documentation officielle, taille de la communauté, ressources tierces.

3. Architecture A — Laravel + Flutter

Vue d'ensemble

Backend : Laravel 11 (PHP 8.4) exposant une API REST documentée par Swagger (L5-Swagger). Authentification par tokens via Laravel Sanctum. Base de données MariaDB.

Front-Office mobile : Flutter (Dart) avec Riverpod pour la gestion d'état et GoRouter pour la navigation. Un seul codebase pour iOS et Android.

Back-Office : Flutter Web — même codebase que mobile, compilé pour navigateur, réutilisant les modèles et la couche service.

Avantages

- **Productivité maximale** : Laravel offre un ORM puissant (Eloquent), du scaffolding (artisan), des form requests, des ressources et une couche de validation déjà mature.
- **Cohérence Front-Office / Back-Office** : Flutter permet de partager les modèles, services HTTP et composants entre mobile et web admin.
- **Sanctum + middleware role** : authentification API simple et adaptée aux SPA.
- **Documentation interactive** : Swagger UI auto-générée à partir d'annotations OpenAPI dans les contrôleurs.
- **Hot-reload** côté Flutter — gain de temps massif sur l'UI.

Inconvénients

- PHP est moins rapide qu'un runtime compilé (Go, Rust) pour les charges très lourdes — non bloquant ici.
- Flutter Web a un temps de chargement initial supérieur à un SPA React/Vue (taille du runtime Dart).
- Compétences Dart parfois rares dans les équipes.

4. Architecture B — Symfony + Vue.js + React Native

Vue d'ensemble

Backend : Symfony 7 (PHP 8.4) avec API Platform pour exposer une API REST/GraphQL automatiquement, JWT authentication via LexikJWTAuthenticationBundle.

Front-Office mobile : React Native (TypeScript) avec Redux Toolkit, navigation via React Navigation.

Back-Office : Vue.js 3 + Vite + Pinia, séparé du Front-Office.

Avantages

- API Platform génère automatiquement les endpoints CRUD et la documentation OpenAPI.
- Vue.js et React Native bénéficient de communautés très larges (recrutement facilité).
- Approche modulaire (Symfony bundles) bien adaptée aux gros projets long terme.
- React Native peut intégrer du code natif iOS/Android pour des cas spécifiques.

Inconvénients

- **Trois technologies distinctes** à maîtriser (PHP/Symfony, JS/TS pour mobile, JS/TS pour web admin) — surcoût de formation.
- React Native nécessite des configurations natives (CocoaPods, Gradle) plus contraignantes que Flutter.
- Pas de partage de code entre mobile et back-office.
- Symfony a une courbe d'apprentissage plus raide que Laravel.

5. Architecture C — Node.js (NestJS) + React + React Native

Vue d'ensemble

Backend : NestJS (TypeScript) avec TypeORM ou Prisma, JWT et Passport.js. Architecture inspirée d'Angular (modules, providers, decorators).

Front-Office mobile : React Native + Expo, TanStack Query pour l'état serveur.

Back-Office : React + Vite + TanStack Router.

Avantages

- **TypeScript de bout en bout** : possibilité de partager les types entre backend, mobile et back-office.
- Performances excellentes côté serveur (Node.js, V8).
- Écosystème NPM très riche (packages pour quasi tout).
- React Native + Expo simplifie le déploiement mobile.

Inconvénients

- Productivité initiale plus faible que Laravel pour les opérations CRUD classiques (plus de boilerplate).
- Choix multiples (TypeORM vs Prisma vs Drizzle, etc.) — décisions à prendre tôt.

- Écosystème JS instable : breaking changes fréquents sur les bibliothèques.
- Pas de scaffolding équivalent à php artisan make:..

6. Synthèse comparative

Le tableau ci-dessous synthétise les notes attribuées à chaque architecture sur les six critères retenus (échelle de 1 à 5).

Critère	A — Laravel + Flutter	B — Symfony + VueJS	C — NextJS + React + GraphQL
Productivité	5	3	3
Mobile cross-platform	5	4	4
Maintenabilité	5	4	4
Performance	4	4	5
Coût (formation incluse)	5	3	3
Documentation & communauté	5	4	5
TOTAL	29 / 30	22 / 30	24 / 30

Tableau 1 — Scoring comparatif des trois architectures candidates

7. Architecture retenue : Laravel + Flutter

À l'issue de la comparaison, l'**architecture A (Laravel + Flutter)** obtient le score le plus élevé (29/30) et est retenue pour la réalisation du projet CESIZen. Les principales raisons de ce choix sont les suivantes :

Productivité prouvée

Laravel propose des conventions claires (*convention over configuration*), un ORM Eloquent qui réduit drastiquement le boilerplate SQL, et une CLI complète (artisan) qui permet de générer migrations, modèles, contrôleurs et tests en quelques secondes. Sur un projet d'évaluation à durée limitée, ce gain de temps est décisif.

Un seul langage côté front (Dart)

Flutter permet de mutualiser entre l'application mobile et le back-office Web :

- La couche modèle (classes Dart, parsing JSON)
- Les services HTTP (Dio + intercepteur Sanctum)
- Une partie des conventions de design (palette, typographie Montserrat)

Un développeur formé à Flutter peut intervenir indifféremment sur le mobile et sur l'admin, ce qui simplifie l'allocation des ressources.

Authentification stateless adaptée

Laravel Sanctum fournit une authentification par tokens parfaitement adaptée aux clients stateless (mobile, SPA). Les tokens peuvent être révoqués individuellement (logout), et la durée de vie est paramétrable via la configuration.

Documentation auto-générée

L5-Swagger lit les annotations OpenAPI directement dans les contrôleurs et les modèles pour produire une documentation interactive accessible à `/api/documentation`. Cela facilite l'intégration

côté client (mobile / admin) et la communication avec le jury d'évaluation.

Conformité RGPD

Le SoftDelete d'Eloquent permet de supprimer un utilisateur sans perdre l'historique (suppression réversible 30 jours), tout en lui rendant ses données invisibles. La suppression définitive (forceDelete) reste possible si l'utilisateur exerce son droit à l'effacement complet.

8. Stack détaillée de la solution retenue

Backend — API Laravel

Composant	Technologie	Version	Rôle
Langage	PHP	8.4	Runtime serveur
Framework	Laravel	11	Routage, ORM, validation, queue
Base de données	MariaDB	11	Persistance relationnelle
Auth API	Sanctum	4	Tokens utilisateur révocables
Documentation	L5-Swagger	8	OpenAPI 3.1 auto-générée
Tests	PHPUnit	11	Tests unitaires + fonctionnels
Conteneurisation	Docker Compose	v2	Reproductibilité dev/prod
Logs	Monolog	3	Journalisation rotative

Front-Office mobile — Flutter

Composant	Technologie	Version	Rôle
SDK	Flutter	≥ 3.5	Framework UI cross-platform
Langage	Dart	3.5	Logique applicative
State	Riverpod	2.5	Gestion d'état réactive
Routage	GoRouter	14	Navigation déclarative + guards
HTTP	Dio	5	Client HTTP avec intercepteurs
Persistance	shared_preferences	2	Stockage clé/valeur (token)
UI	google_fonts (Montserrat)	6	Typographie
HTML	flutter_html	3	Rendu des pages d'information
Charts	fl_chart	0.68	Visualisation des données

Back-Office — Flutter Web

Le back-office utilise exactement la même stack Flutter que l'application mobile, compilée pour navigateur via la plateforme Web officielle de Flutter (canvaskit ou html). Les différences concrètes se limitent au thème (palette plus sobre, layout desktop avec sidebar) et au routage (admin-only guard).

9. Architecture logicielle de l'API

L'API suit le pattern **MVC enrichi** par une couche service et une couche repository, afin d'isoler la logique métier des préoccupations de persistance et de transport HTTP.

Diagramme des couches

Couche	Responsabilité	Exemples de classes
Routes	Définition des endpoints HTTP, association avec contrôleurs	<code>routes/api.php</code>
Controllers	Réception requête, validation via <code>FormRequest</code> , sérialisation de la réponse	<code>AuthenticationController</code> , <code>EmotionJournalController</code>
FormRequests	Validation des entrées (présence, format, règles métier complexes)	<code>RegistrationRequest</code> , <code>JournalStoreRequest</code>
Services	Logique métier, orchestration, transactions	<code>EmotionJournalService</code> , <code>UserService</code>
Repositories	Accès aux données, requêtes Eloquent complexes	<code>JournalEntryRepository</code> , <code>UserRepository</code>
Models (Eloquent)	Représentation ORM, relations, accessors	<code>User</code> , <code>Emotion</code> , <code>JournalEntry</code>
Resources	Sérialisation JSON sortie (forme réponse API)	<code>UserResource</code> , <code>JournalEntryResource</code>

Avantages de cette architecture

- **Testabilité** : la couche service peut être testée unitairement en mockant les repositories.
- **Inversion des dépendances** : les contrôleurs reçoivent les services via injection (`RepositoryServiceProvider`), pas d'instanciation directe.
- **Réutilisabilité** : un même service peut être appelé par plusieurs contrôleurs (ex. `EmotionJournalService` côté API et côté commande artisan).
- **Lisibilité** : un nouveau développeur retrouve immédiatement où se trouve quoi (logique métier dans `Services/`, accès aux données dans `Repositories/`).

10. Sécurité

Authentification et autorisation

- **Sanctum** : tokens stockés en base (`personal_access_tokens`), révocables individuellement.
- **Bcrypt** : mots de passe hashés via le cast hashed automatique d'Eloquent.
- **Middleware role** personnalisé : restreint l'accès aux routes admin aux comptes ayant le rôle correspondant.
- **Vérification d'état** : les utilisateurs *inactifs* ou *bannis* ne peuvent pas se connecter (HTTP 403).

Protection des données

- **SoftDeletes** sur la table `users` : suppression réversible, conformité RGPD.
- **Cloisonnement** : un utilisateur ne peut accéder qu'à ses propres entrées de journal (vérification `id_user` dans le repository).
- **HTTPS** obligatoire en production (configuration côté reverse-proxy).
- **CORS** configuré strictement en production pour n'autoriser que les origines de confiance.

11. Évolutions envisagées

La stack retenue permet plusieurs évolutions sans rupture architecturale :

- **Notifications push** : intégration de Firebase Cloud Messaging côté Flutter et envoi côté Laravel via le canal fcm.
- **Tests E2E** : ajout de Playwright sur le back-office et de `integration_test` côté Flutter.
- **CI/CD** : pipelines GitHub Actions pour exécuter php artisan test et flutter test à chaque pull request.
- **Multilangue** : le module flutter_localizations est déjà intégré ; ajout de fichiers ARB pour l'anglais.
- **Mode hors-ligne** : cache local des entrées de journal via drift ou hive avec synchronisation différée.

12. Conclusion

L'architecture **Laravel + Flutter** retenue offre le meilleur compromis productivité/maintenabilité/coût pour le projet CESIZen. Elle permet de tenir les délais d'évaluation tout en livrant une plateforme complète, documentée, testée et conforme aux exigences du référentiel CDA Bloc 2.

L'ensemble des choix techniques décrits dans ce document est mis en œuvre dans les trois sous-projets livrés (api/, mobile/, backoffice/) et démontré par le code source, les tests automatisés et la documentation Swagger interactive.